

Introduction to Computer Science

Fall 2026

Homework 2

Due: October 4, 2026 at 23:58:59
No Copy-and-Paste. No Late Submission.

Instructions. Please write your answers to each problem using L^AT_EX and submit the compiled PDF file. If you find a problem difficult to understand or solve, you may use external resources, including search engines (such as Google), online references (such as Wikipedia), and AI tools (such as DeepSeek), to study relevant concepts and assist your learning. However, you are responsible for understanding your submitted solution, and **please make sure every word in your submission is written by yourself to avoid plagiarism**. If you use any AI tool while working on the assignment, please include a short disclosure at the end of your submission stating which tool you used and briefly describing how you used it. Undisclosed use of AI tools may be subject to point deductions.

1 Graph Basics (3 points)

An undirected graph consists of a set of *vertices* and a set of *edges*, where each edge connects two vertices. A *path* is a sequence of vertices in which every two consecutive vertices are connected by an edge. A *cycle* is a path that starts and ends at the same vertex, with no other repeated vertices. A *connected component* is a maximal set of vertices in which every pair of vertices is connected by a path.

- (a) One common way to represent a graph in a computer is by its *adjacency matrix*. For a graph with vertices $v_1, \dots, v_n$, its adjacency matrix $A \in \{0, 1\}^{n \times n}$ is defined by

$$A_{ij} = \begin{cases} 1, & \text{if there is an edge between } v_i \text{ and } v_j, \\ 0, & \text{otherwise.} \end{cases}$$

Consider the graph G on vertices $\{a, b, c, d, e, f, g\}$ with adjacency matrix

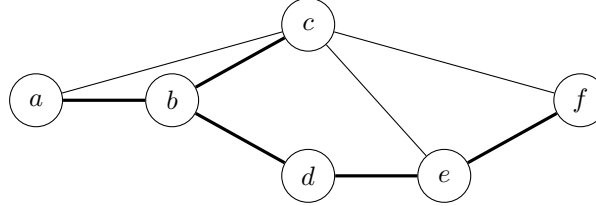
	a	b	c	d	e	f	g
a	0	0	0	1	0	1	0
b	0	0	0	0	1	0	0
c	0	0	0	0	0	0	1
d	1	0	0	0	0	1	0
e	0	1	0	0	0	0	0
f	1	0	0	1	0	0	0
g	0	0	1	0	0	0	0

List all connected components of G and give one cycle in G .

More generally, suppose an undirected graph has k connected components and we are allowed to add arbitrary edges. What is the minimum number of new edges required to make the graph connected? Briefly justify your answer, and give one set of new edges that achieves this minimum for G .

- (b) A *tree* is a connected graph that contains no cycle. A *spanning tree* of a connected graph H is a subgraph that contains every vertex of H and is itself a tree.

Consider the graph H below. The **bold** edges form a subgraph T ; the remaining edges belong to H but not to T .



First determine whether T is a spanning tree of H .

In T , find the unique simple path from c to f . Now add the edge cf to T . Give the cycle that is created. If we require the edge cf to remain, list all edges of the original tree T that can be removed so that the resulting graph is again a spanning tree of H .

2 Algorithm and Time complexity (4 points)

This problem introduces basic concepts in algorithm complexity analysis and priority queue data structures. Read the following background introductions carefully before answering the questions.

Algorithm 1 Median-of-Medians Selection

Input: Array A and integer k

Output: The k -th smallest element of A

```

1: if  $|A| \leq 5$  then
2:   | Sort  $A$  directly
3:   | return the  $k$ -th element of  $A$ 
4: Divide  $A$  into groups of five elements
5: Find the median of each group and store them in  $M$ 
6:  $x \leftarrow \text{SELECT}(M, \lceil |M|/2 \rceil)$ 
7: Partition  $A$  into  $A_{<}, A_{=}, A_{>}$  according to pivot  $x$ 
8: if  $k \leq |A_{<}|$  then
9:   | return  $\text{SELECT}(A_{<}, k)$ 
10: else if  $k \leq |A_{<}| + |A_{=}|$  then
11:   | return  $x$ 
12: else
13:   | return  $\text{SELECT}(A_{>}, k - |A_{<}| - |A_{=}|)$ 

```

Asymptotic Notation: The running time of an algorithm is usually described as a function of the input size n . When analyzing algorithms, we are mainly interested in the growth rate as n becomes large. Therefore, we ignore constant factors and lower-order terms. For two positive functions $f(n)$ and $g(n)$, we write $f(n) = O(g(n))$ if there exist positive constants c and n_0 such that $f(n) \leq cg(n)$ for all $n \geq n_0$. Thus, $O(\cdot)$ describes an asymptotic upper bound. Similarly, we write $f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that $f(n) \geq cg(n)$ for all $n \geq n_0$. Thus, $\Omega(\cdot)$ describes an asymptotic lower bound. Finally, we write $f(n) = \Theta(g(n))$ if there exist positive constants c_1 , c_2 , and n_0 such that $c_1g(n) \leq f(n) \leq c_2g(n)$ for all $n \geq n_0$. Intuitively, $f(n) = \Theta(g(n))$ means that $f(n)$ and $g(n)$ grow at the same asymptotic rate. Equivalently, $f(n) = \Theta(g(n))$ holds when both $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.

Median Selection ($\Theta(n)$): Given an unsorted array A of n distinct numbers, the median is the element that would appear in the middle position after sorting. More generally, the k -th smallest element can be found without sorting the entire array. The algorithm 1, known as Median-of-Medians, finds the k -th smallest element in deterministic linear time.

Sorting ($\Theta(n \log n)$): Sorting is a fundamental operation in computer science. For comparison-based sorting algorithms, the worst-case lower bound is $\Omega(n \log n)$ comparisons. MergeSort illustrated by algorithm 2 achieves this bound.

Algorithm 2 Merge Sort

Input: An array A

Output: An array containing the elements of A in increasing order

1: **if** $|A| \leq 1$ **then**

2: $\perp$ **return** A

3: $n \leftarrow |A|$

4: $m \leftarrow \lfloor n/2 \rfloor$

5: $L \leftarrow \text{MERGESORT}(A[1 : m])$

6: $R \leftarrow \text{MERGESORT}(A[m + 1 : n])$

7: $B \leftarrow \text{MERGE}(L, R)$

8: **return** B

$\triangleright$ Merge L and R in increasing order
 $\triangleright$ The merge step takes $\Theta(|L| + |R|)$ time

(a) Answer the following questions.

- (1) For the input array $A = [8, 3, 10, 1, 6, 4, 9]$, write down the output of the sorting algorithm 2 and the value returned by the median algorithm 1 with $k = 4$.
- (2) Suppose that the numbers of basic operations performed by the two algorithms are approximated by $T_{\text{median}}(n) = 20n$, $T_{\text{sort}}(n) = 2n \log_2 n$. These formulas are simplified models for this exercise. You may use them directly without analyzing the internal details of the algorithms. If $n = 2^{30}$, which algorithm is estimated to perform fewer basic operations, and by what factor? Then find the value of n at which the two simplified models predict the same number of basic operations.

(b) A *data structure* is a way of organizing and storing data so that different operations can be performed efficiently. Different data structures may provide different trade-offs between memory usage and operation time.

A *priority queue* is a type of data structure that maintains a set of elements, where each element has an associated key representing its priority. Unlike a regular queue, elements are removed according to their priority rather than their insertion order. Common operations include **Insert**, **Find-Min**, **Delete-Min**, and **Decrease-Key**.

Different implementations of priority queues provide different performance guarantees. For example:

- A **Binary Heap** maintains elements in a complete binary tree satisfying the heap property. It supports **Insert**, **Delete-Min**, and **Decrease-Key** in $O(\log n)$ worst-case time.
- A **Binomial Heap** is a collection of binomial trees. A binomial tree B_k is defined recursively: B_0 contains one node, and B_k is formed by connecting two copies of B_{k-1} by making the root of one tree a child of the root of the other tree. A binomial heap stores at most one binomial tree of each degree. Binomial heaps support efficient merging of two priority queues. The standard implementation supports **Insert**, **Merge**, and **Delete-Min** in $O(\log n)$ worst-case time.
- A **Fibonacci Heap** is a collection of heap-ordered trees and delays some structural updates until necessary. It supports **Insert** and **Decrease-Key** in $O(1)$ amortized time, and **Delete-Min** in $O(\log n)$ amortized time.

Suppose a priority queue is initially empty. We perform n **Insert**, m **Decrease-Key**, n **Delete-Min** operations. Compute the asymptotic running time of this sequence of operations when using binary heap and Fibonacci heap respectively.

- (c) Prim's algorithm can be implemented with different priority queue data structures. Suppose that the graph is represented using adjacency lists and has n vertices and m edges.

Assume that the implementation performs at most n **insert** operations, n **delete-min** operations, and m **decrease-key** operations. Scanning all adjacency lists takes $O(m)$ time.

Now suppose that the priority queue is implemented using a Fibonacci heap. Assume that **insert** and **decrease-key** take amortized $O(1)$ time, while **delete-min** takes amortized $O(\log n)$ time. Derive the total amortized running time of Prim's algorithm using the Fibonacci heap for the priority queue.

Yao's algorithm for the minimum spanning tree problem was introduced in class and is described in [1]. Compare it with the Fibonacci heap implementation of Prim's algorithm in the following cases:

- (1) $m = \Theta(n)$;
- (2) $m = \Theta(n \log n)$.

For each of the case, determine which algorithm has the asymptotically smaller running-time bound.

3 Shortest Path (3 points)

Alice is riding a bicycle from her dormitory to a lecture hall. The campus road network is shown in Figure 1. Each road is undirected, and the number beside a road is its length. The labeled vertices S, A, B, C, D, E, F, T represent different locations on campus. The diagram is schematic and is not drawn to scale.

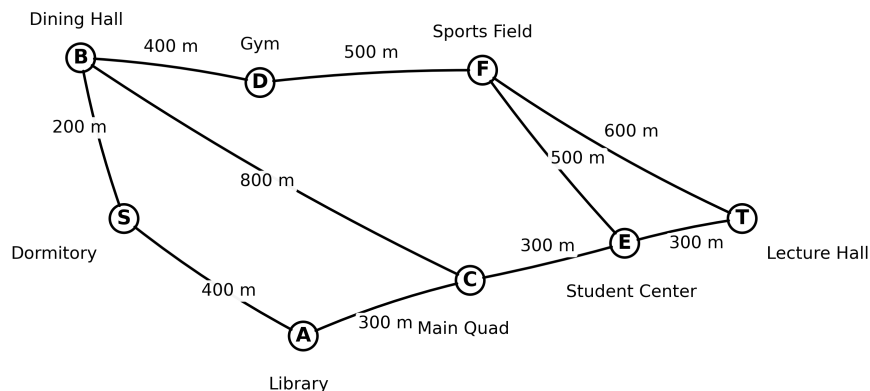


Figure 1: A campus road network.

(a) **Dijkstra's algorithm.** Alice first wants to minimize the total distance traveled. Run Dijkstra's algorithm from the Dormitory S and stop when the Lecture Hall T is finalized. Give the order in which the vertices are finalized, and give the shortest route from S to T together with its total distance.

(b) **A guided search.** Dijkstra's algorithm chooses the unfinalized vertex with the smallest distance already traveled from S . For a map, however, we also have information about where the destination is.

For each vertex v , let $g(v)$ be the shortest distance from S to v found so far, and let $h(v)$ be the straight-line distance from v to T . The straight-line distances for this campus are:

v	S	A	B	C	D	E	F	T
$h(v)$ (m)	1000	700	1200	450	930	200	540	0

Consider a search algorithm that is identical to Dijkstra’s algorithm, except that it chooses the unfinalized vertex with the smallest

$$f(v) = g(v) + h(v)$$

instead of the smallest $g(v)$. This is the central idea of the A^* (*A-star*) algorithm [2].

Run A^* from S and stop when T is finalized. Give the order in which the vertices are finalized, and state how many vertices are finalized in total.

(c) From shortest distance to shortest time. Alice now cares about travel time rather than distance. She can ride at at most 20 km/h. During the rush hour, however, the road from the Main Quad C to the Student Center E becomes heavily congested, and Alice can travel on that road at only 5 km/h. All other roads can still be traveled at 20 km/h.

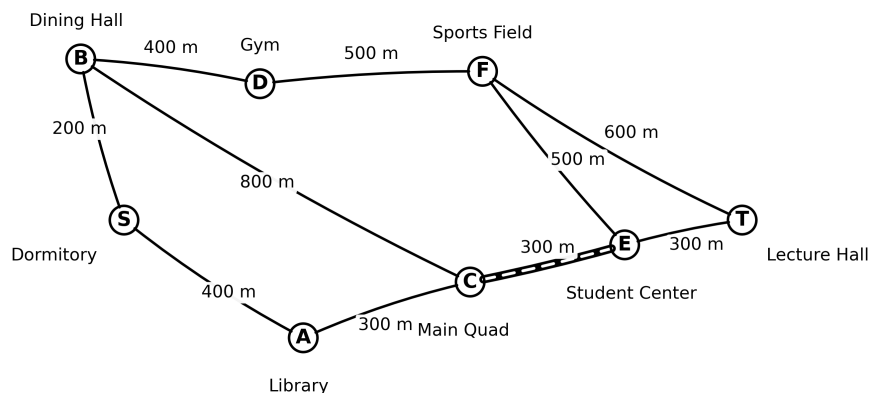


Figure 2: The same campus during the rush hour.

A heuristic is called *admissible* if it never overestimates the minimum possible remaining cost. In this problem, you may use without proof the fact that an admissible heuristic can be used by A^* while preserving the shortest-path guarantee. If you are interested, you may think about why this is true or look up A^* on your own.

Using the straight-line distance from v to T , design an admissible heuristic for the *remaining travel time*. Briefly justify why your heuristic never overestimates the minimum remaining travel time. Then run A^* using your heuristic and stop when T is finalized. Give the order in which the vertices are finalized, and give Alice’s fastest route from S to T together with its total travel time.

Declaration of AI Use

Please disclose any use of generative AI tools in completing this homework. For each tool used, briefly state what you used it for. You remain responsible for the correctness and originality of your submission. If you did not use any generative AI tools, remove this section.

References

- [1] Andrew Chi chih Yao. An $O(|E| \log \log |V|)$ algorithm for finding minimum spanning trees. *Information Processing Letters*, 4(1):21–23, 1975.
- [2] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.